Arrays

Content notes, classwork and homework with worked C++ solutions.

Lesson 1: Arrays - Intro

Quick recap: std::vector basics • Indexing • Reading input into an array • Printing an array

Theory & Nuances

What is an array?

An array is a row of boxes that all hold the same type of value and sit next to each other in memory. Instead of making 100 separate variables m1, m2, ..., m100 for 100 marks, you make ONE array that holds all 100 and reach each box by its position. Together with loops, this is what lets a program handle any amount of data with the same few lines of code.

std::vector basics

In C++ the easiest array to use is std::vector from the <vector> header. vector<int> arr = {10, 20, 30}; makes a vector with three ints. vector<int> arr(n); makes n boxes that all start at 0, and vector<int> arr(n, 7); makes n boxes that all start at 7. arr.size() tells you how many boxes there are, and arr.push_back(x) adds a new box with x at the end, so a vector can grow while the program runs. The older C-style array int arr[5]; has a fixed size and does not know its own length, which is why we prefer vector.

Indexing

Boxes are numbered from 0, not 1. In an array of size n the first element is arr[0] and the last is arr[n - 1]. Reading or writing arr[i] takes the same O(1) time whatever i is, because the computer jumps straight to the box. Using an index outside 0..n-1 (for example arr[n] or arr[-1]) is undefined behaviour: the program may print garbage, crash, or seem to work and fail later. C++ does not check it for you; arr.at(i) does check, and stops the program with an error instead.

Reading input into an array

First read how many values are coming (n), then create the vector with that size, then fill it with a loop: for (int i = 0; i < n; i++) cin >> arr[i];. cin skips spaces and newlines, so the values can be typed on one line or many. If you do not know n in advance, start with an empty vector and use push_back for each value you read.

Printing an array

Printing is the same loop with cout: for (int i = 0; i < n; i++) cout << arr[i] << " ";. Changing the loop changes what you print: start at n - 1 and count down to print in reverse, or step by 2 (i += 2) to print only the even positions. The range-based loop for (int x : arr) cout << x << " "; is a shorter way to visit every element in order when you do not need the index.

Key Points / Common Mistakes

- Indexes start at 0: the last element of a size-n array is arr[n - 1], not arr[n].
- A loop over an array runs while i < n, never i <= n - that reads one box past the end.
- vector<int> arr(n) gives n zeros; vector<int> arr; is EMPTY, so arr[0] on it is an error.
- arr.size() returns an unsigned number; store it in an int (int n = arr.size();) before doing n - 1 maths.
- Read n first, then create the vector of size n, then read the n values.
- Use for (int x : arr) to just visit values; use an index loop when you need the position i.

Classwork Problems

1. Read N values and print them

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cout << "How many values? ";
    cin >> n;

    vector<int> arr(n);                 // n boxes, all start at 0
    cout << "Enter " << n << " values: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];                  // fill box i
    }
    cout << endl;

    cout << "You entered: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    cout << "First element arr[0]    = " << arr[0] << endl;
    cout << "Last element arr[n - 1] = " << arr[n - 1] << endl;
    return 0;
}
```

Output (typed input: 5 12 7 25 3 18)

```
How many values? Enter 5 values:
You entered: 12 7 25 3 18
First element arr[0]    = 12
Last element arr[n - 1] = 18
```

2. Print in reverse order by index

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> arr = {10, 20, 30, 40, 50};
    int n = arr.size();

    cout << "Original order: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    cout << "Reverse order : ";
    for (int i = n - 1; i >= 0; i--) {  // start at the last index
        cout << arr[i] << " ";
    }
    cout << endl;

    for (int i = n - 1; i >= 0; i--) {
        cout << "arr[" << i << "] = " << arr[i] << endl;
    }
    return 0;
}
```

Output

```
Original order: 10 20 30 40 50
Reverse order : 50 40 30 20 10
arr[4] = 50
arr[3] = 40
arr[2] = 30
arr[1] = 20
arr[0] = 10
```

3. Print elements at even indices

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> arr = {5, 8, 13, 21, 34, 55, 89};
    int n = arr.size();

    cout << "Elements at even indices:" << endl;
    for (int i = 0; i < n; i += 2) {    // 0, 2, 4, 6
        cout << "  index " << i << " -> " << arr[i] << endl;
    }

    cout << "Elements at odd indices: ";
    for (int i = 1; i < n; i += 2) {    // 1, 3, 5
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Output

```
Elements at even indices:
  index 0 -> 5
  index 2 -> 13
  index 4 -> 34
  index 6 -> 89
Elements at odd indices: 8 21 55
```

Homework Problems

1. Sum of an array

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> arr = {4, 9, 1, 7, 3};
    int n = arr.size();
    int sum = 0;

    for (int i = 0; i < n; i++) {
        sum += arr[i];
        cout << "add " << arr[i] << " -> running sum = " << sum << endl;
    }
    cout << "Sum of the array = " << sum << endl;
    return 0;
}
```

Output

```
add 4 -> running sum = 4
add 9 -> running sum = 13
add 1 -> running sum = 14
add 7 -> running sum = 21
add 3 -> running sum = 24
Sum of the array = 24
```

2. Count positive elements

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> arr = {3, -1, 0, 7, -5, 2, 0, 9};
    int count = 0;

    cout << "Positive elements: ";
    for (int x : arr) {
        if (x > 0) {                    // 0 is NOT positive
            cout << x << " ";
            count++;
        }
    }
    cout << endl;
    cout << "Count of positive elements = " << count << endl;
    return 0;
}
```

Output

```
Positive elements: 3 7 2 9
Count of positive elements = 4
```

Lesson 2: Array Traversal

Quick recap: Sum and average • Maximum and minimum • Counting with a condition

Theory & Nuances

Traversal = visiting every element once

Traversing an array means walking through it from index 0 to n - 1 and doing something with each element. Almost every array problem starts this way. A traversal touches each of the n elements once, so it takes O(n) time. While you walk, you keep a few helper variables (a running sum, the best value so far, a counter) that are updated at every step and hold the answer at the end.

Sum and average

Start with sum = 0 and add each element. Use long long if the values or the count are large, because a sum of many ints can overflow. The average is sum / n, but if both are ints this is INTEGER division and the decimal part is thrown away (389 / 5 = 77). Convert first: (double)sum / n gives 77.8. Print it with fixed << setprecision(2) from <iomanip> to control the decimal places.

Maximum and minimum

Start the best-so-far with the FIRST element: mx = arr[0]; mn = arr[0]; and then compare every other element. Do not start the maximum at 0 - if every value is negative, 0 would wrongly win. You can find max and min together in a single pass with two if statements in the same loop. To remember WHERE the maximum is, store its index too.

Second largest

Keep two variables, largest and second. When a new value beats largest, the old largest becomes second. Otherwise, if it beats only second (and is not equal to largest), it becomes the new second. One pass is enough; sorting the whole array would also work but costs O(n log n) instead of O(n).

Counting with a condition

Keep a counter starting at 0 and add 1 whenever the element passes a test: if (x > 0) count++;, if (x % 2 == 0) evens++;. Some questions need two passes: to count elements above the average you must first know the average (pass 1), and only then compare each element with it (pass 2). Two passes is still O(n).

Key Points / Common Mistakes

- Initialise max/min with arr[0], not 0 - an array of negatives breaks the 0 trick.

- sum / n with two ints is integer division; use (double)sum / n for the real average.
- Use long long for sums when values or n are big, to avoid overflow.
- Negative odd numbers give x % 2 == -1, so test evens with x % 2 == 0, not odds with x % 2 == 1.
- For second largest, handle duplicates: {35, 35, 34} has second largest 34, not 35.
- Each traversal is O(n); two separate passes are still O(n).

Classwork Problems

1. Max and min in one pass

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> arr = {34, 7, 89, 12, 56, 3, 71};
    int n = arr.size();

    int mx = arr[0], mn = arr[0];       // start with the first element
    int maxIdx = 0, minIdx = 0;
    for (int i = 1; i < n; i++) {
        if (arr[i] > mx) {
            mx = arr[i];
            maxIdx = i;
        }
        if (arr[i] < mn) {
            mn = arr[i];
            minIdx = i;
        }
    }
    cout << "Maximum = " << mx << " at index " << maxIdx << endl;
    cout << "Minimum = " << mn << " at index " << minIdx << endl;
    return 0;
}
```

Output

```
Maximum = 89 at index 2
Minimum = 3 at index 5
```

2. Average of the array

```
#include <iostream>
#include <iomanip>
#include <vector>
using namespace std;

int main() {
    vector<int> marks = {78, 85, 92, 64, 70};
    int n = marks.size();

    int sum = 0;
    for (int x : marks) {
        sum += x;
    }
    cout << "Sum = " << sum << endl;
    cout << "sum / n (integer division) = " << sum / n << endl;

    double avg = (double)sum / n;       // convert BEFORE dividing
    cout << fixed << setprecision(2);
    cout << "Average = " << avg << endl;
    return 0;
}
```

Output

```
Sum = 389
sum / n (integer division) = 77
Average = 77.80
```

3. Count elements above the average

```
#include <iostream>
#include <iomanip>
#include <vector>
using namespace std;

int main() {
    vector<int> arr = {12, 45, 7, 30, 18, 60};
    int n = arr.size();

    // pass 1: find the average
    int sum = 0;
    for (int x : arr) {
        sum += x;
    }
    double avg = (double)sum / n;
    cout << fixed << setprecision(2) << "Average = " << avg << endl;

    // pass 2: count elements greater than the average
    int count = 0;
    cout << "Above average: ";
    for (int x : arr) {
        if (x > avg) {
            cout << x << " ";
            count++;
        }
    }
    cout << endl;
    cout << "Count = " << count << endl;
    return 0;
}
```

Output

```
Average = 28.67
Above average: 45 30 60
Count = 3
```

Homework Problems

1. Second largest element

```
#include <iostream>
#include <climits>
#include <vector>
using namespace std;

int main() {
    vector<int> arr = {12, 35, 1, 10, 34, 35};

    int largest = INT_MIN, second = INT_MIN;
    for (int x : arr) {
        if (x > largest) {
            second = largest;           // old largest moves down
            largest = x;
        } else if (x > second && x != largest) {
            second = x;
        }
    }
    cout << "Largest        = " << largest << endl;
    if (second == INT_MIN) {
        cout << "No second largest (all values equal)" << endl;
    } else {
        cout << "Second largest = " << second << endl;
    }
    return 0;
}
```

Output

```
Largest        = 35
Second largest = 34
```

2. Count even and odd

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> arr = {7, 12, 5, 18, 21, 4, 9};
    int evens = 0, odds = 0;

    for (int x : arr) {
        if (x % 2 == 0) {
            evens++;
        } else {
            odds++;
        }
    }
    cout << "Even count = " << evens << endl;
    cout << "Odd count  = " << odds << endl;
    cout << "Total      = " << evens + odds << endl;
    return 0;
}
```

Output

```
Even count = 3
Odd count  = 4
Total      = 7
```